

Grille d'autoévaluation - Semaine 2

420-W30-SF · Tests unitaires, AAA, TDD, doublures manuelles et Moq · Automne 2026

Nom (facultatif)

Date de révision

Comment utiliser cette grille

Cachez vos notes, puis cochez seulement ce que vous pouvez expliquer ou réaliser sans suivre une recette. Commencez votre révision par une case non cochée.

1. Solutions, projets et dépendances

Je suis capable de :

- ☐ distinguer une solution Visual Studio d'un projet .NET;
- ☐ créer une solution avec un projet de production et un projet de tests;
- ☐ ajouter la référence du projet de tests vers le projet de production;
- ☐ distinguer un ProjectReference d'un PackageReference;
- ☐ installer Moq seulement dans le projet de tests;
- ☐ restaurer, compiler et tester la solution depuis Visual Studio ou avec dotnet.

Élément de la structure que je dois revoir

2. Tests unitaires et structure AAA

Je suis capable de :

- ☐ formuler le comportement précis qu'un test doit vérifier;
- ☐ séparer clairement Arranger, Agir et Auditer;
- ☐ déclencher une seule opération principale dans Agir;
- ☐ vérifier un résultat observable et précis dans Auditer;
- ☐ nommer un test selon Methode_Cas_ResultatAttendu;
- ☐ appliquer FIRST et reconnaître un test fragile ou dépendant d'un autre test;
- ☐ choisir des données minimales et une assertion assez précise.

Nom d'un test AAA que je peux écrire sans aide

3. Tests paramétrés et exceptions

Je suis capable de :

- ☐ transformer des exemples d'une même règle en [Theory];
- ☐ choisir entre [InlineData] et MemberData, notamment pour des decimal;
- ☐ tester une valeur normale, une frontière valide et une valeur invalide;
- ☐ interpréter séparément l'échec de chaque jeu de données;
- ☐ vérifier une levée avec Assert.Throws<T>();
- ☐ vérifier le type précis de l'exception et son ParamName;
- ☐ distinguer une exception anormale d'un refus métier attendu.

Cas frontière ou invalide que je dois retester

4. Cycle TDD

Je suis capable de :

- ☐ expliquer, de mémoire, le rôle de chacune des trois étapes du cycle TDD;
- ☐ écrire un petit test qui échoue pour la bonne raison;
- ☐ confirmer l'échec avant de modifier le code de production;
- ☐ écrire le minimum de code nécessaire pour obtenir le vert;
- ☐ réusiner seulement lorsque tous les tests sont verts;
- ☐ distinguer TDD de l'ajout de tests après l'implantation.

Petite règle que je pourrais développer en TDD

5. Dépendances et doublures

Je suis capable de :

- ☐ repérer une dépendance qui rend un test lent, fragile ou imprévisible;
- ☐ décider quand conserver un objet réel simple et déterministe;
- ☐ distinguer une vérification d'état final d'une vérification d'interaction;
- ☐ reconnaître « doublure » comme terme général;
- ☐ distinguer substitut, espion, fausse implantation et simulacre;
- ☐ choisir une doublure selon ce que le test doit observer;
- ☐ expliquer pourquoi doubler chaque objet fragilise les tests.

Dépendance que je doublerais, ou garderais réelle, et raison

6. Simulacre manuel et Moq

Je suis capable de :

- ☐ écrire une classe de test qui implante une interface de dépendance;
- ☐ mémoriser puis vérifier le nombre d'appels et les paramètres reçus;
- ☐ vérifier qu'aucun appel n'a lieu dans un scénario de refus;
- ☐ expliquer ce que Moq automatise par rapport au simulacre manuel;
- ☐ créer un `Mock<T>` et fournir `.Object` au système testé;
- ☐ préparer une réponse avec `Setup(...).Returns(...)`;
- ☐ vérifier une interaction avec `Verify`, `Times.Once` ou `Times.Never`;
- ☐ contraindre un argument avec une valeur exacte ou `It.Is<T>`;
- ☐ utiliser `VerifyNoOtherCalls()` après les vérifications principales;
- ☐ expliquer oralement la différence entre `Setup` et `Verify`.

Interaction que je pourrais vérifier avec Moq

7. Défis de vérification

Sans consulter immédiatement les notes, je peux :

- ☐ écrire un test AAA d'un calcul déterministe;
- ☐ convertir trois exemples semblables en une théorie;
- ☐ tester une exception et son `ParamName`;
- ☐ effectuer un petit cycle rouge-vert-réusinage;
- ☐ écrire une doublure manuelle qui vérifie un appel et ses paramètres;
- ☐ tester avec Moq qu'un paiement refusé empêche l'envoi en cuisine.

8. Mon plan de révision

Notion prioritaire et prochaine action concrète

Question à apporter en classe